

AI와 함께 개발하기

채팅에서 하네스까지 — 신뢰를 쌓는 3단계



NDS 사내 개발자 교육 · 180분 핸드온

플랫폼 개발실 유원상

[2026.07.09]



프롬프트로 → 영상 한 편

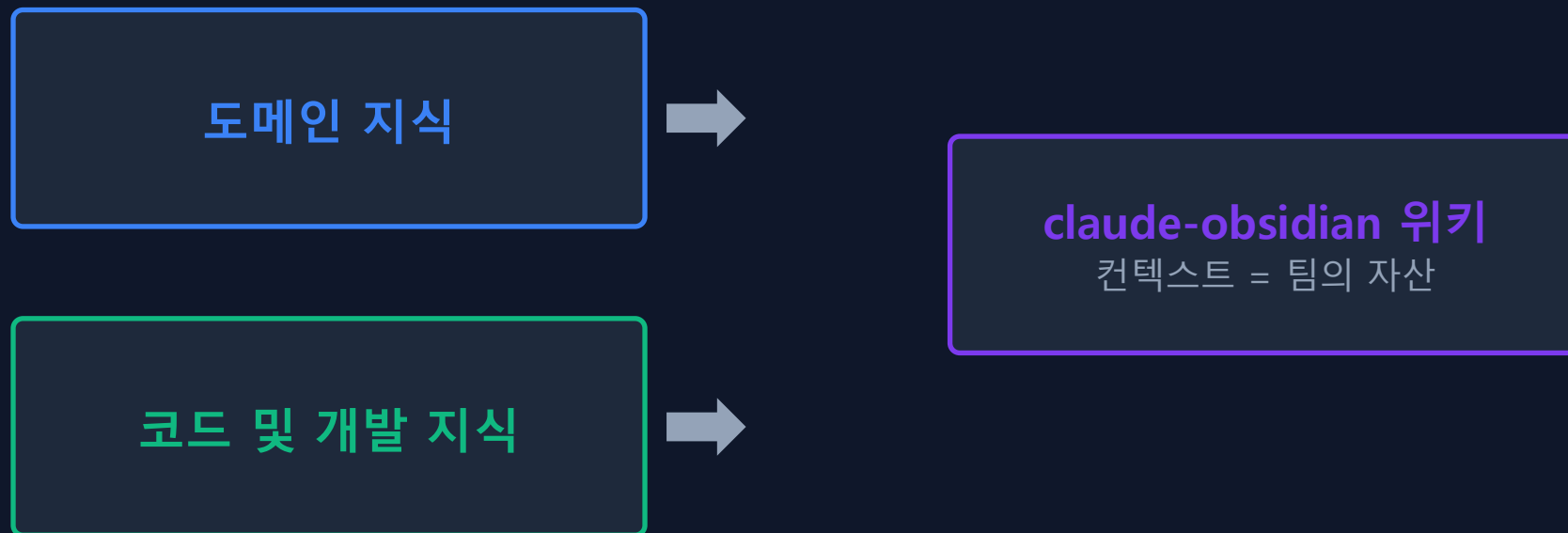
Gemini 라이브 시연 — 지금 직접 만들어 보겠습니다

<https://github.com/softallice/ai-edu.git>

AI와 쌓은 지식을 어떻게 자산화할 것인가?

● ① 코드와 도메인을 자산으로 만들기

업무나 개발을 진행하면서 쌓인 노하우와 기술, 방법을 자산으로 만들기



핵심: 코드는 1회성, 위키는 누적 자산 — 두 축을 같이 굴린다.

어떻게 설치하나?

wsl

cd ~

cd /mnt/c/Users

cd [개인폴더]

git clone https://github.com/AgriciDaniel/claude-obsidian.git

cd claude-obsidian

bash bin/setup-vault.sh

claude

/plugin marketplace add /mnt/c/Users/[개인폴더]/claude-obsidian

/plugin install claude-obsidian@agricidaniel-claude-obsidian

/reload-plugins

● ② 엔지니어링 위키 (Obsidian · claude-obsidian)

무엇을 쓰나

claude-obsidian = Claude Code 플러그인 겸
Obsidian vault (Karpathy LLM Wiki 패턴).

ingest 하면 교차 참조된 위키 페이지가 자동 생성되고,
query 하면 학습데이터가 아니라 '우리가 쌓은 위키'에서 답합니다.

지식이 세션마다 복리로 늘어납니다.

vault 구조	
.raw/	소스 원본 (읽기 전용)
wiki/	자동 생성 지식 베이스
index.md	전체 색인
hot.md	최근 컨텍스트 캐시
log.md	작업 로그
domain/	도메인 규칙
schema/	테이블 명세
standards/	코딩·보안 표준
_templates/	entity·concept·source

● ② claude-obsidian 명령 & 흐름

명령	하는 일
<code>/wiki</code>	셋업·스캐폴드, '이 vault 용도?' 1회 질문 후 구조 생성
<code>ingest</code> [파일]	.raw/ 소스 → 교차참조 위키 페이지 8~15개 + index/log 갱신
<code>query:</code> [질문]	위키 내용으로 답변(근거 페이지 인용)
<code>/save</code> [이름]	현재 대화를 구조화된 위키 노트로 저장
<code>lint the wiki</code>	고아·끊긴 링크·공백 점검 (ingest 10~15회마다)

실습 흐름

bash bin/setup-vault.sh → Obsidian에서 폴더 열기 → `/wiki` → `.raw/`에 DDL·엔티티·표준 투입 → `ingest` → `query`로 검증

신기하죠. 그런데 —

프롬프트로(채팅)만 개발을 할 수 있을까?

오늘의 질문 — AI 코드를 어떻게 활용하고 신뢰할 것인가

180분 + 60분, 이렇게 갑니다.

1. 인트로 및 위키 설정— 방금 본 것 (10분 + 40분)

2. AI 개발 3단계 (15분)

3. ✂ 실습 1 · 환경 구축 (30분)

4. ✂ 실습 2 · 소스 클론·실행 (15분) + ☕ 10분

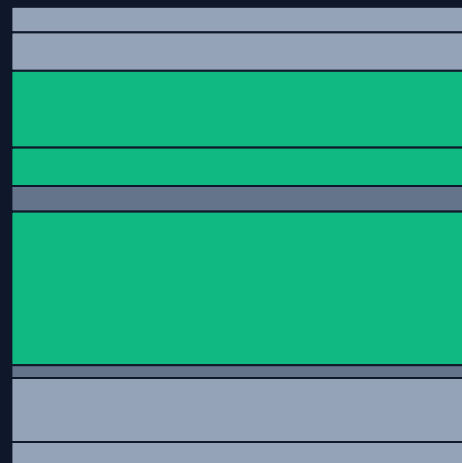
5. ✂ 실습 3 · IDE 미션 5종 (60분) + ☕ 5분

6. 하네스 — 왜 필요한가 (25분)

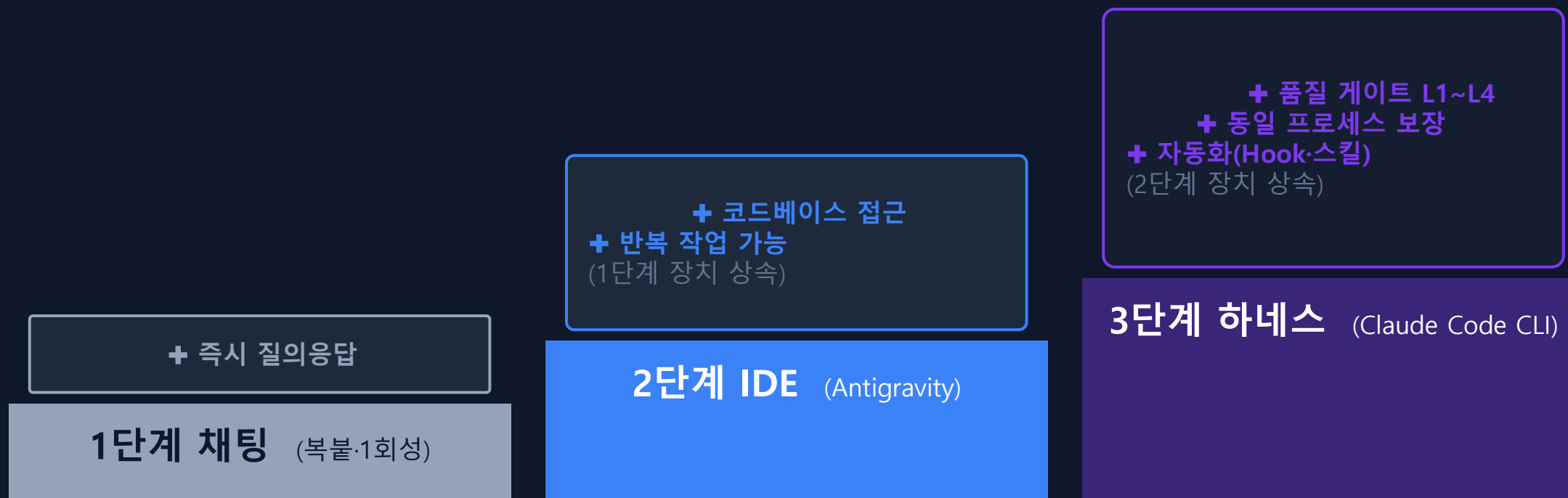
7. 마무리 + Q&A (10분)

60%

오늘 시간의 60%는
여러분 키보드 위에서 (105/180분)



단계가 오를수록, 신뢰 장치가 쌓인다



장치 개수 **1개 → 3개 → 6개** — 단계가 오를 때마다 장치가 2배

3단계는 후반부에 다시 옵니다

● 1단계 · 채팅 — “다들 해보셨죠?”

좋았던 것

- + 즉시 질의응답
- + 진입장벽 제로
- + 프롬프트(코드) 조각·단발 질문에 최강

그런데...

복불 지옥 — 소스 나르기의 반복
새 창마다 컨텍스트 리셋
결과는 1회성, 재현 불가

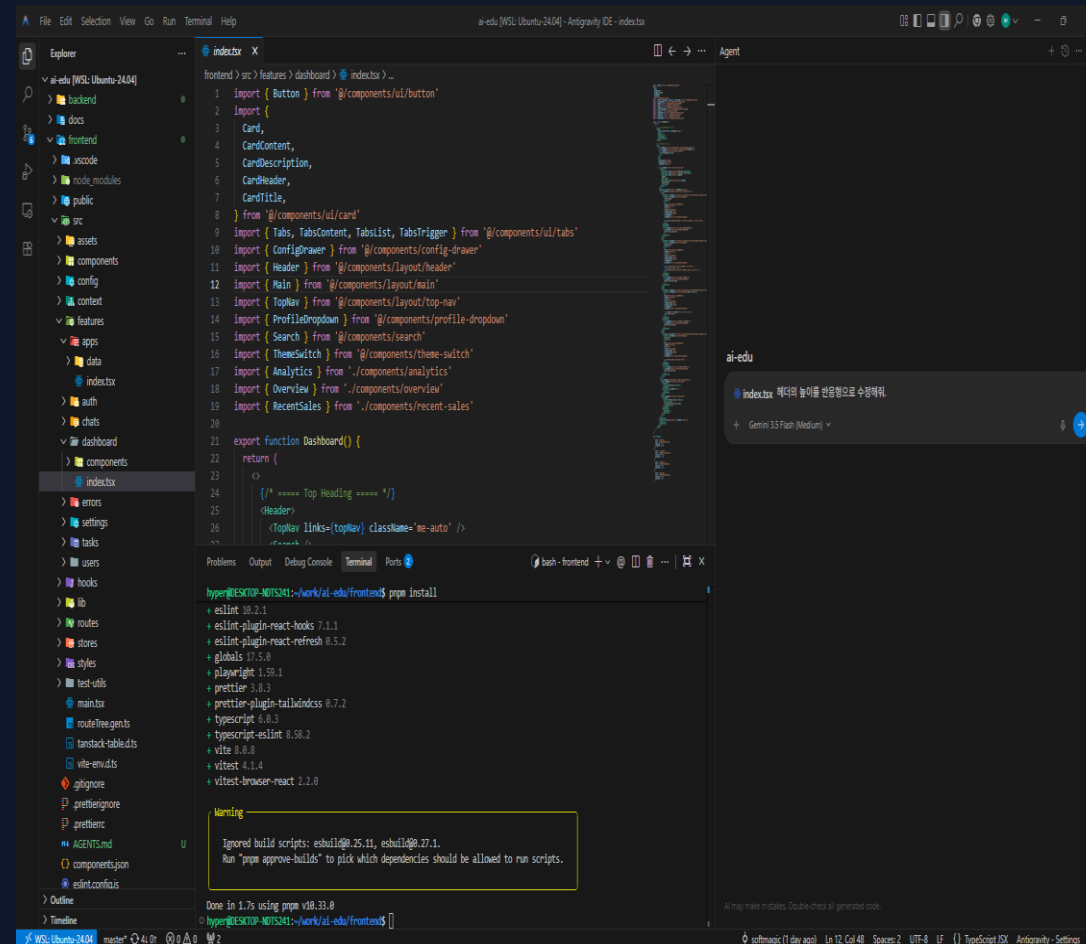
● 2단계 · IDE — AI가 코드베이스 안으로 들어온다

복붙 없이 코드베이스 직접 접근

수정 → 확인 → 재요청, 반복 작업 가능

멀티모달 입력(이미지·문서)

오늘 실습 3의 무대 — 60분간 여기서 미션 4개를 수행합니다.



● 3단계 · 하네스 — 신뢰를 시스템으로

🔒 컨텍스트 유지

🔒 동일 프로세스

🔒 품질 게이트

“이게 왜 필요한지는 —
여러분이 직접 느낀 뒤에 다시 이야기합니
다.”

```
$ claude |
```

작업의 크기에 따라, 필요한 장치가 다르다

구 분	1단계 채팅	2단계 IDE	3단계 하네스
컨텍스트 유지	○ 매번 복붙, 새 창마다 리셋	● 코드베이스 접근, 세션 단위	● 문서·메모리로 세션 간 유지
재현성	○ 사람·시점마다 결과 상이	● 같은 IDE라도 프롬프트 의존	● 동일 프로세스 보장
품질 통제	○ 전적으로 사람 눈	● 사람 리뷰 + IDE 보조	● 품질 게이트 L1~L4 자동 검증
적합 작업	단발 질문·코드 조각	화면 단위 개발·수정	마이그레이션·반복 대량 작업

✂ 실습 1 · 30분 (00:25-00:55)

환경 구축 — 10분 투자로 AI 개발 환경 완성

사전 설치 회신자는 **빠른 확인 열만** 수행 → 끝나면 옆자리를 도와주세요

☐	설치 항목	용도	빠른 확인
☐	WSL + Ubuntu	리눅스 개발 환경	<code>wsl -l -v</code>
☐	Antigravity IDE	실습 3의 무대 (IDE 채팅 개발)	앱 실행
☐	VSCode	보조 에디터	<code>code --version</code>
☐	Claude Code CLI	3단계 하네스의 입구	<code>claude --version</code>
☐	Node.js · JAVA · Python	프론트/백 런타임	<code>node -v · python --version</code>

상세 명령은 다음 두 장 +

<https://github.com/softallice/ai-edu/blob/master/docs/ai-실습/0.사전설치가이드.md>

① WSL + Ubuntu — 관리자 권한으로

① 관리자 PowerShell에서 (시작 메뉴 → 우클릭 → 관리자 권한)

```
wsl --install
```

② 재부팅 → Ubuntu 첫 실행 → 사용자명/암호 설정

③ 확인

```
wsl -l -v
```

→ Ubuntu Running 2 ← 초록 글씨가 보이면 성공

범례: 흰색 = 입력 · 초록 = 기대 출력

② 도구 4종 — 위에서 아래로 순서대로

Antigravity IDE

<https://antigravity.google/download>

-> 윈도우용 다운로드

VSCode

<https://code.visualstudio.com/>

`code --version` → 버전이 보이면 성공

Claude Code CLI

<https://code.claude.com/docs/en/overview>

`claude --version` → 버전이 보이면 성공

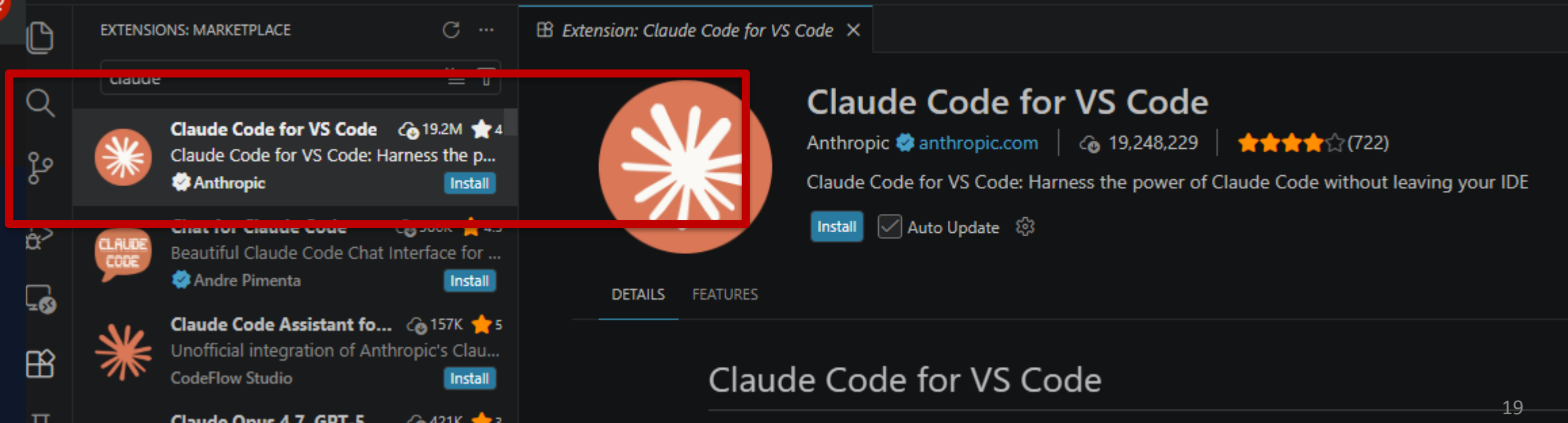
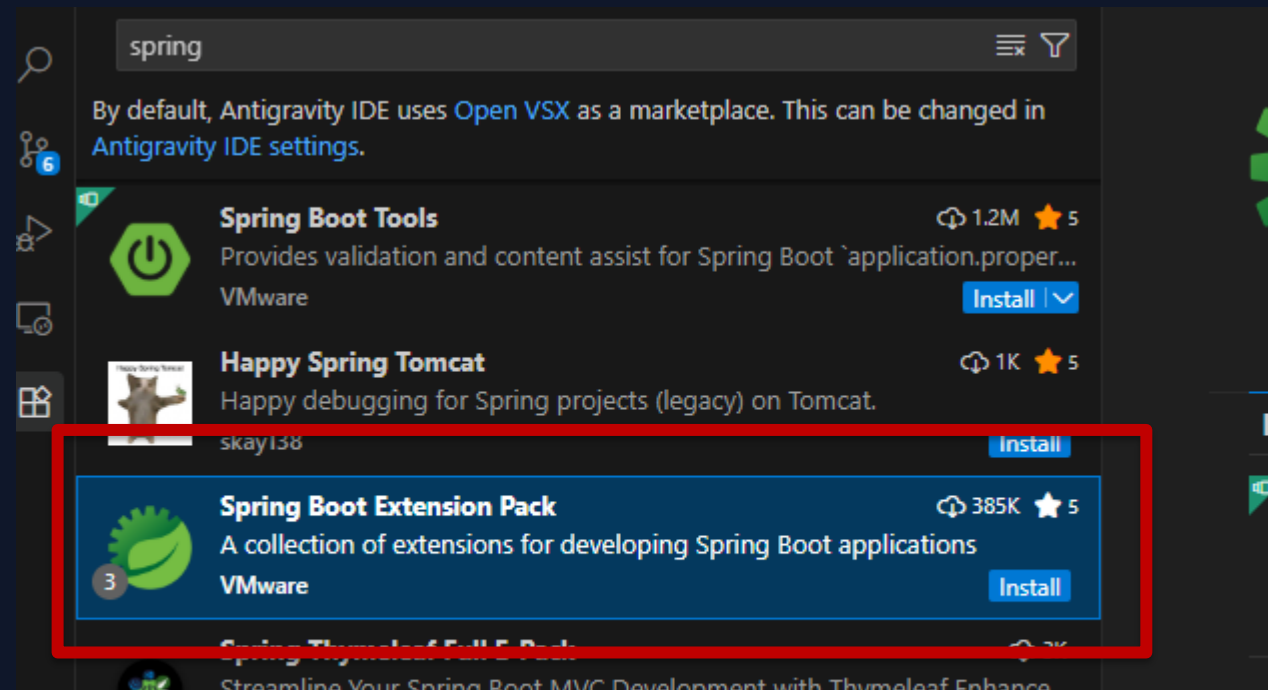
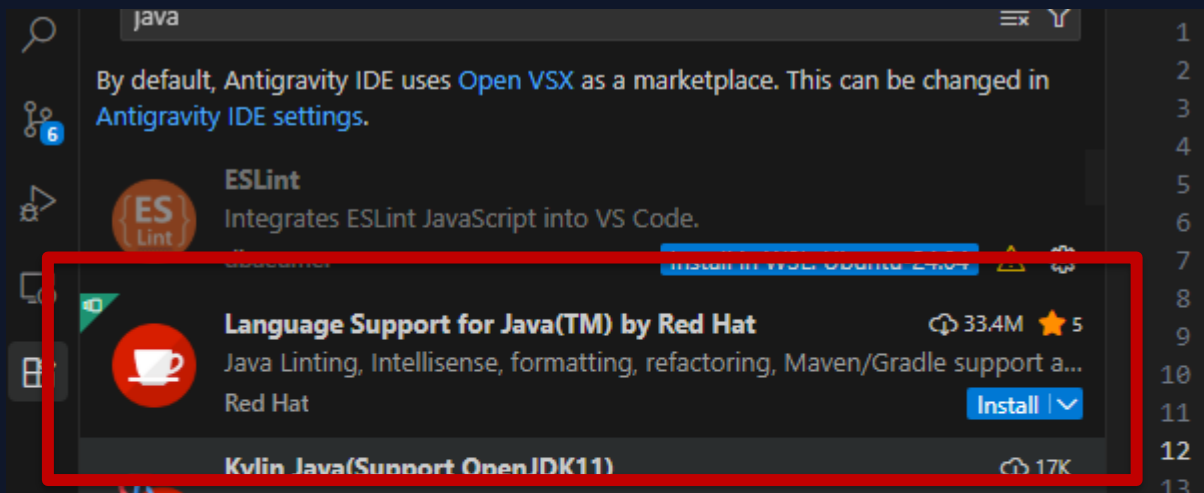
Node.js · Python

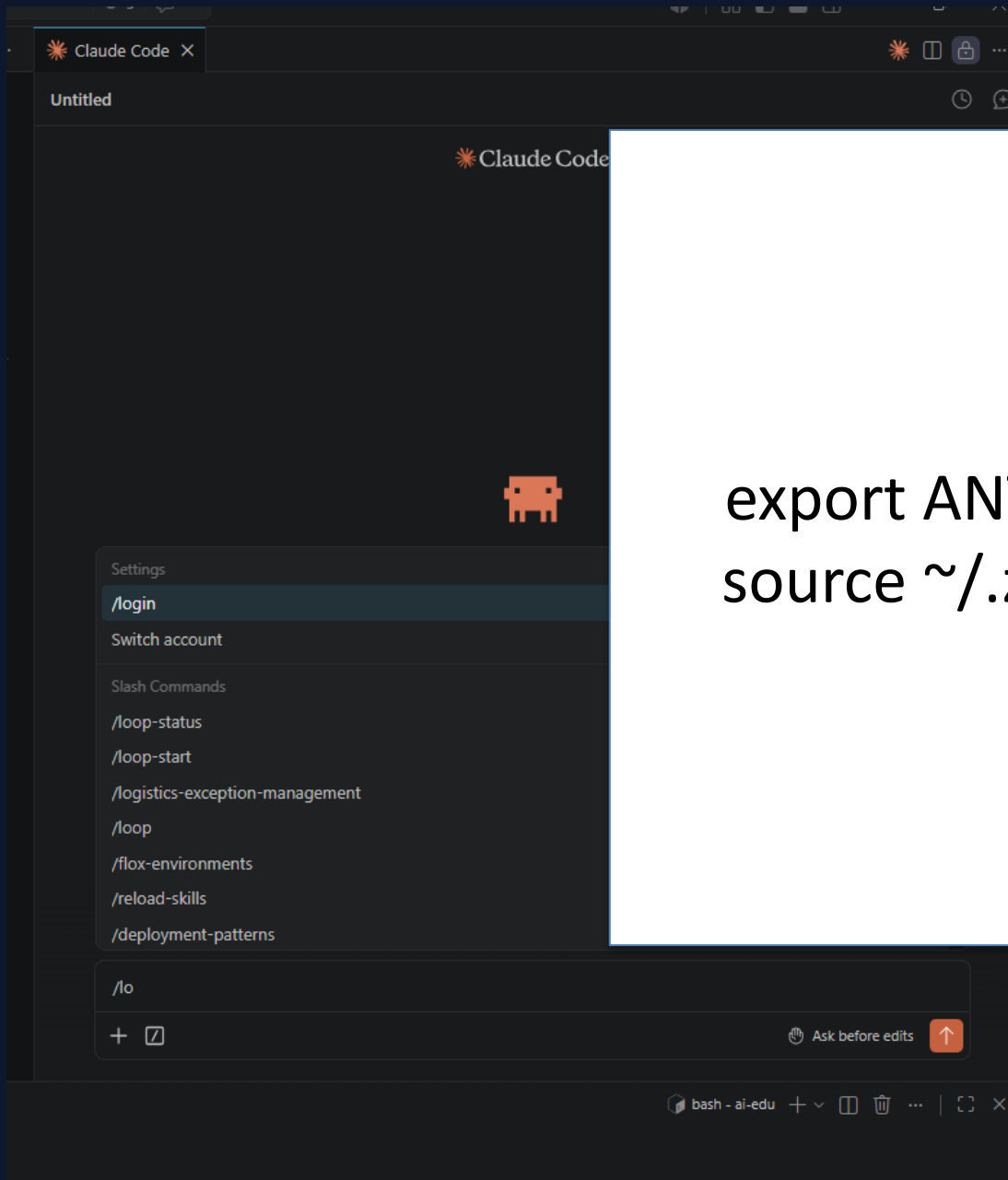
`sudo apt-get install -y nodejs`

`node -v` · `python --version`

막히면 다음 장 — 사내망 3대 함정

IDE에 Extensions 설치





```
export ANTHROPIC_API_KEY="your_api_key_here"  
source ~/.zshrc
```

막혔다면 — 내 증상부터 찾으세요

증상 (이게 보이면)	원인	해결
SSL certificate problem / UNABLE_TO_VERIFY_LEAF	사내망 TLS 가로채기 인증서 - 농심 인증서 등록	사내 루트 인증서 등록 WSL의 우분투에서
다운로드 멈춤 / ETIMEDOUT	프록시 미설정	export HTTPS_PROXY= △[사내 프록시 주소]
wsl --install 실패 / 가상화 오류	관리자 권한·BIOS 가상화 비활성	관리자 PowerShell 재실행

“3분 이상 막히면: 손 들기 🙋 → 옆자리 페어”

세 줄이 보이면 통과

입력할 명령	이게 보이면 통과 ✓
<code>claude --version</code>	2.1.173 (Claude Code)
<code>node -v</code>	v24.14.1
<code>java --version</code>	openjdk 25.0.2 2026-01-20 LTS OpenJDK Runtime Environment Corretto- 25.0.2.10.1 (build 25.0.2+10-LTS) OpenJDK 64-Bit Server VM Corretto- 25.0.2.10.1 (build 25.0.2+10-LTS, mixed mode, sharing)

✂ 실습 2 · 15분 (00:55-01:10)

공개 GitHub에서 클론 — 내 노트북에서 풀스택 띄우기

실습 리포: **ai-edu** 템플릿 — React-TS(프론트) + Spring Boot(백엔드)

목표 1: 프론트엔드 실행 → 브라우저에서 화면 확인

목표 2: Java 백엔드 구성 확인 → 기동

<https://github.com/softallice/ai-edu>

좌: 프론트 / 우: 백엔드 — 둘 다 띄웁니다

프론트엔드 (React-TS)

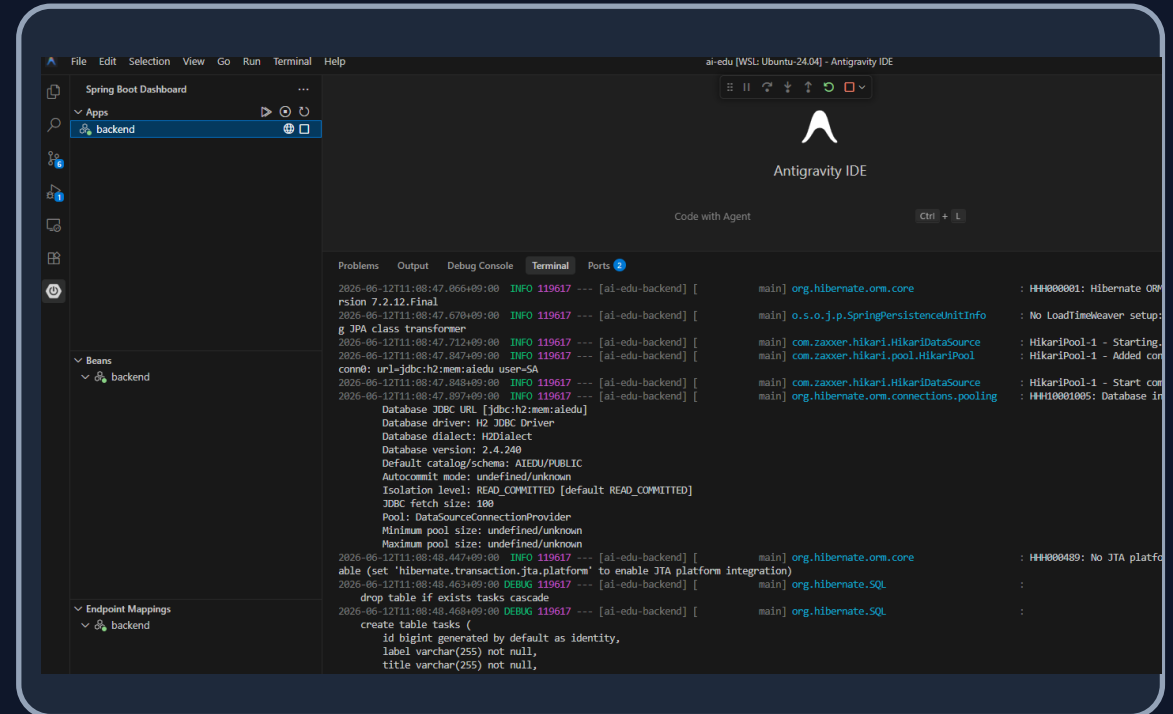
```
mkdir work  
cd work
```

```
git clone https://github.com/softallice/ai-edu
```

```
cd frontend
```

```
pnpm install && pnpm dev
```

```
→ http://localhost:5173
```



팁: pnpm install 중 SSL 에러 → S13 표 1행(인증서) 참조

✓ CHECKPOINT 2 — 풀스택 기동

확인	이게 보이면 통과 ✓
브라우저 localhost:5173	실습 앱 첫 화면이 뜬다
백엔드 콘솔	<code>Started ...Application</code>

“지금 개발 환경의 설정을 마쳤습니다 ”

✂ 실습 3 (핵심) · 60분 (01:20-02:20)

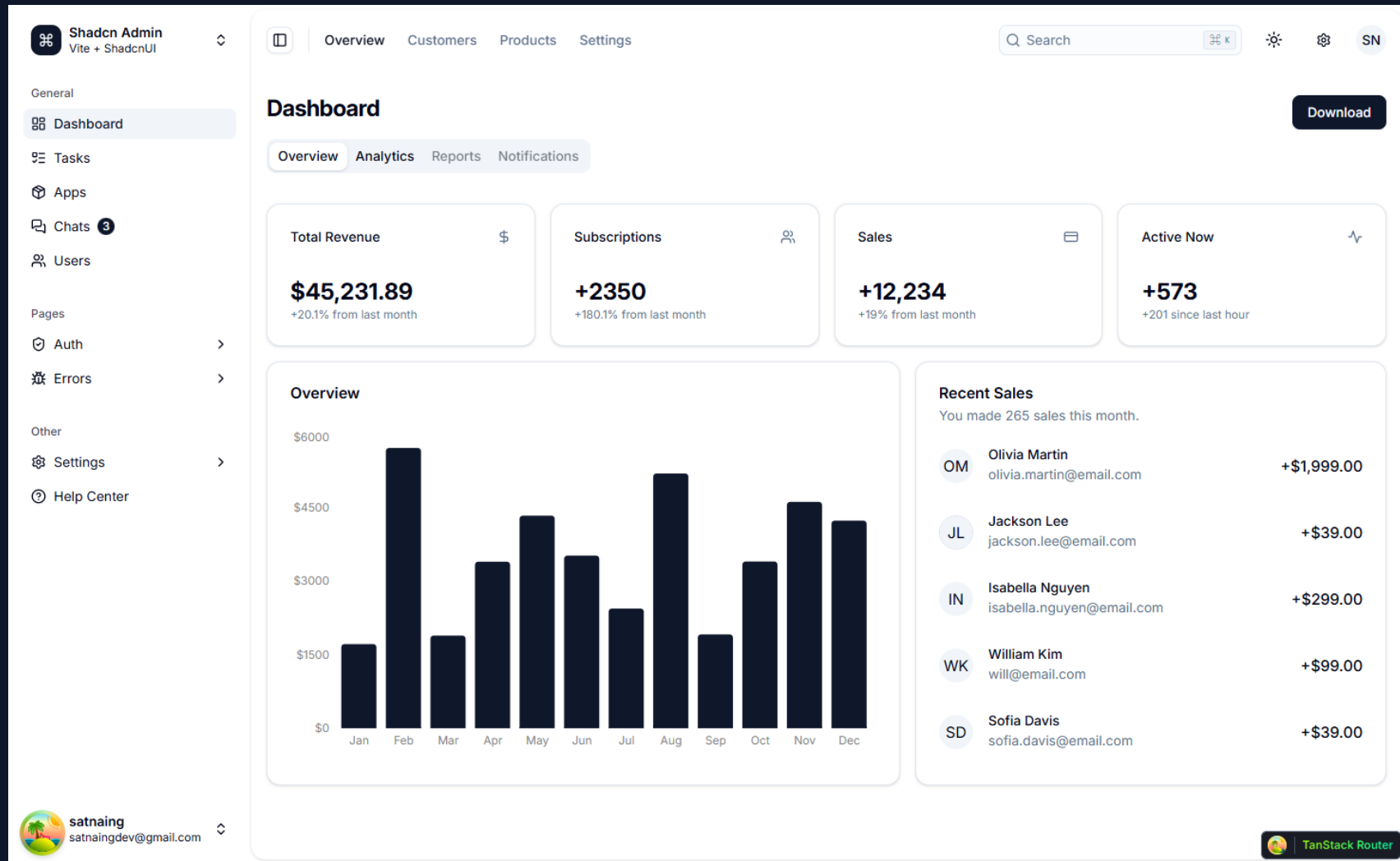
미션 5개 — 입력이 점점 자유로워집니다



각 10~12분 권장 · 총 60분 (개요·회고 포함, 01:20-02:20) — 난이도 ▲ = 입력의 자유도

"12분짜리 성공 5번 — 미션 클리어 방식으로 갑니다"

● 실행 화면 localhost:5173



● 미션 1 — 기존 화면 수정

목표

`http://localhost:5173/tasks`

기존 목록 화면에 컬럼 1개를 추가한다 (예: '등록일')

체크포인트

브라우저 목록 화면에 새 컬럼이 보인다

프롬프트 / 명령어

초안: "src/[목록 화면 파일 경로]를 열어
테이블에 '등록일' 컬럼을 추가해줘.
날짜 형식은 YYYY-MM-DD."

막혔을 때

- ① 프롬프트에 파일 경로 명시
- ② 새 채팅으로 재시도
- ③ 손 들기 / 옆자리 페어

● 미션 1 — 기존 화면 수정 결과

Tasks
Here's a list of your tasks for this month!

Filter by title or ID... ⊕ Status ⊕ Priority 🔍 View

☐ Task	Title	Status	Priority	
☐ TASK-9366	Documentation Auctus bardus minus pariat... vobis solitudo tamquam solitudo.	🗑 Canceled	↓ Low	...
☐ TASK-5736	Bug Admoneo vehemens suscipit toties des... idero tollo allatus blanditiis caute delibe...	🗑 Canceled	→ Medium	...
☐ TASK-3204	Documentation Deputo veritas vinculum expedita casus supplant... corona deserunt calamitas considero soleo coma t...	🗑 Canceled	↑ High	...
☐ TASK-7141	Bug Vester ducimus aequus minima possimus vilis cuppedia celo alter depereo.	✅ Done	↑ High	...
☐ TASK-8689	Documentation Admoveo nihil denique acer corrumpo cupio peccatus spectaculum tumultus tergum cui thalassinus v...	🕒 Backlog	→ Medium	...
☐ TASK-3359	Feature Carus minus uberrime crapula damnatio tristis correptus adhaero itaque defendo.	🗑 Canceled	↑ High	...
☐ TASK-4715	Bug Solutio cohaero baiulus brevis animadverto adfero adeo callide calco quibusdam vapulus tergum.	🗑 Canceled	→ Medium	...
☐ TASK-7138	Feature Usitas tardus aliquid comprehendo cupiditas a patria statim copiose crux.	✅ Done	↓ Low	...
☐ TASK-1373	Feature Peior alias comedo pel avertio stultus caries.	🗑 Canceled	↓ Low	...
☐ TASK-4140	Feature Eius bibo vulgaris cenaculum sponte est.	✅ Done	↓ Low	...

10 Rows per page Page 1 of 10 << < 1 2 3 4 ... 10 > >>

Tasks
Here's a list of your tasks for this month!

Filter by title or ID... ⊕ Status ⊕ Priority 🔍 View

☐ Task	Title	Status	Priority	등록일	
☐ TASK-9366	Documentation Auctus bardus minus pariat... vobis solitudo tamquam solitudo.	🗑 Canceled	↓ Low	2025-07-21	...
☐ TASK-5736	Bug Admoneo vehemens suscipit toties des... idero tollo allatus blanditiis caute delibe...	🗑 Canceled	→ Medium	2025-08-12	...
☐ TASK-3204	Documentation Deputo veritas vinculum expedita casus supplant... corona deserunt calami...	🗑 Canceled	↑ High	2025-07-21	...
☐ TASK-7141	Bug Vester ducimus aequus minima possimus vilis cuppedia celo alter depereo.	✅ Done	↑ High	2026-03-01	...
☐ TASK-8689	Documentation Admoveo nihil denique acer corrumpo cupio peccatus spectaculum tumult...	🕒 Backlog	→ Medium	2026-04-30	...
☐ TASK-3359	Feature Carus minus uberrime crapula damnatio tristis correptus adhaero itaque defendo.	🗑 Canceled	↑ High	2025-11-30	...
☐ TASK-4715	Bug Solutio cohaero baiulus brevis animadverto adfero adeo callide calco quibusdam vap...	🗑 Canceled	→ Medium	2025-08-23	...
☐ TASK-7138	Feature Usitas tardus aliquid comprehendo cupiditas a patria statim copiose crux.	✅ Done	↓ Low	2026-06-12	...
☐ TASK-1373	Feature Peior alias comedo pel avertio stultus caries.	🗑 Canceled	↓ Low	2025-09-23	...
☐ TASK-4140	Feature Eius bibo vulgaris cenaculum sponte est.	✅ Done	↓ Low	2026-03-27	...

10 Rows per page Page 1 of 10 << < 1 2 3 4 ... 10 > >>

채팅 프롬프트 : frontend tasks에서 등록일 컬럼 추가해줘

● 미션 2 — 이미지로 화면 만들기

목표

화면 시안 이미지 1장을 주고 그대로 구현한다

체크포인트

구현 화면이 시안과 같은 레이아웃으로 렌더된다

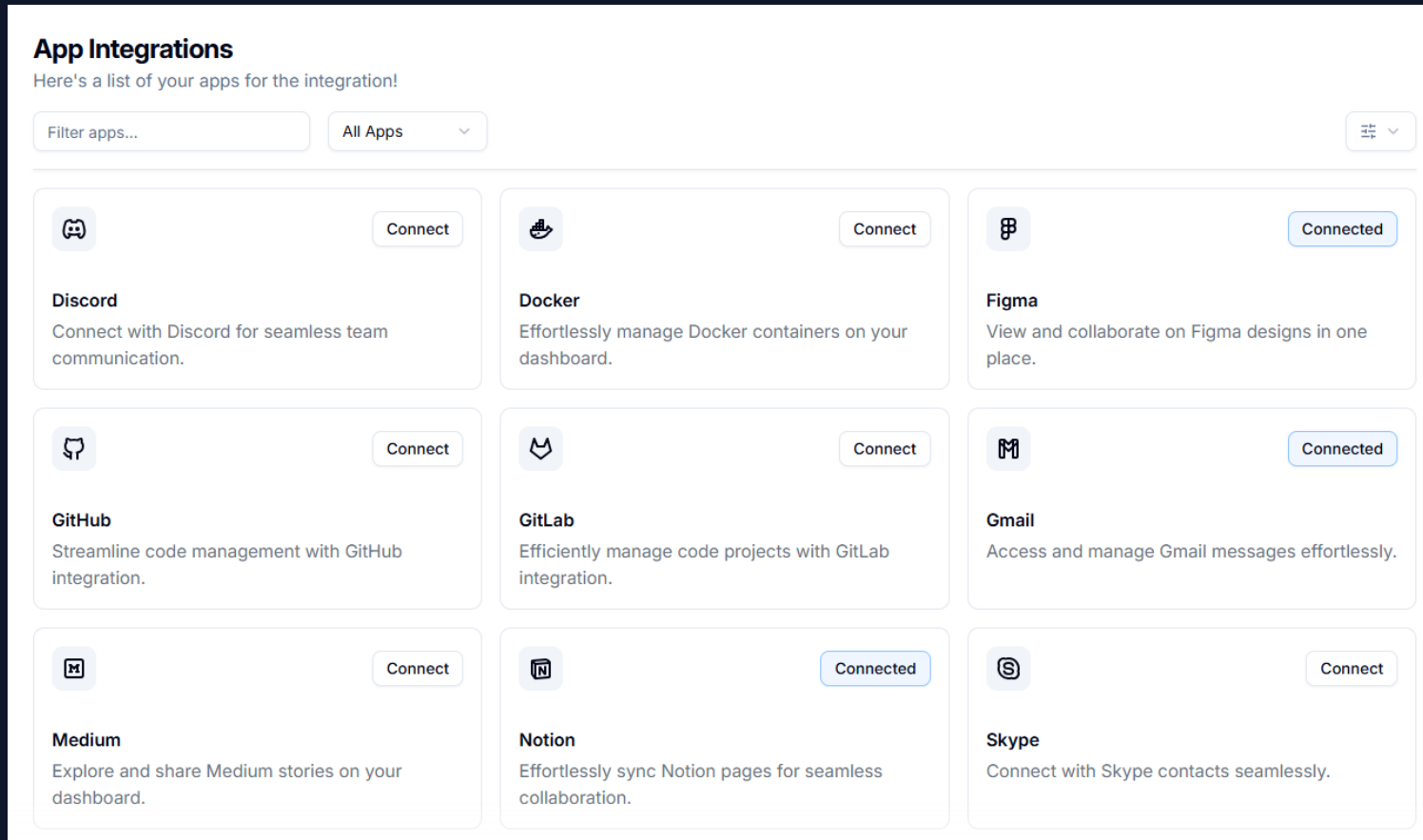
프롬프트 / 명령어

- ① 시안 다운로드: ai-edu/docs/ai-실습/13.미션2-이미지_화면_만들기.png
- ② IDE 채팅에 이미지 첨부 후:
"첨부한 시안과 동일한 레이아웃의 화면을
React 컴포넌트로 만들어줘.
기존 스타일 시스템을 따라줘."

막혔을 때

- ① 이미지가 첨부됐는지 확인
- ② "시안의 △[특징 1개]를 꼭 반영해줘"로 재시도
- ③ 페어 / 손

● 미션 2 — 이미지로 화면 만들기 결과



● 미션 3 — 프롬프트만으로 화면 만들기

💬 “옆 사람과 화면이 같나요?”

🎯 목표

이미지 없이 텍스트 요구사항만으로 신규 화면을 만든다

✅ 체크포인트

신규 화면이 뜨고 저장 → 목록 이동이 동작한다

📄 프롬프트 / 명령어

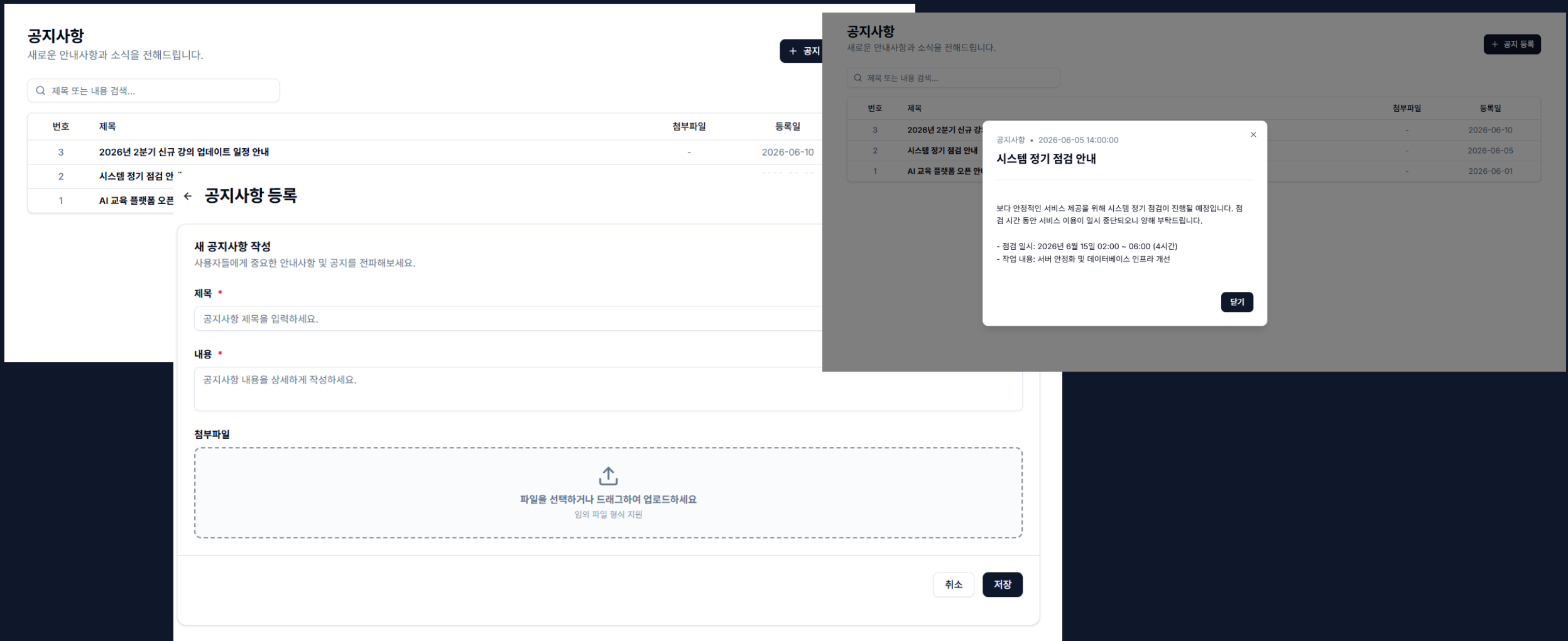
△[미션 3 예시 프롬프트 - 초안:]

"공지사항 등록 화면을 새로 만들어줘.
제목·내용·첨부파일 입력,
저장/취소 버튼, 저장 시 목록으로 이동."

SOS 막혔을 때

- ① 요구사항을 3줄로 쪼개 다시
- ② 새 채팅으로 재시도
- ③ 페어 / 손

● 미션 3 — 프롬프트만으로 화면 만들기 결과



● 미션 4 — 설계서(md) 기반 개발

◆ 미션 3과 비교해 보세요 — 결과가 안정적이지 않나요?

🎯 목표

설계서 md를 먼저 쓰고, 그 문서로 화면을 만든다

✅ 체크포인트

설계서에 적은 항목이 빠짐없이 화면에 반영됐다

📄 프롬프트 / 명령어

- ① design.md 작성 (템플릿 제공):
화면명 / 입력 필드 표 / 동작 규칙 3줄
[△\[ai-edu/docs/ai-실습/12.design.md\]](#)
- ② 채팅에: "design.md를 읽고
그대로 화면을 구현해줘."

🆘 SOS 막혔을 때

- ① 설계서에 누락 항목 추가 후 "다시 읽고 수정해줘"
- ② 새 채팅 + 설계서 첨부 재시도
- ③ 페어 / 손

● 미션 4 — 설계서(md) 기반 개발 화면

Shadcn Admin
Vite + ShadcnUI

General

Dashboard

Tasks

Notices

Apps

Chats 3

Users

Customers

Pages

Auth >

Errors >

Other

Settings >

Help Center

Q Search

K

고객 관리

고객사 정보를 등록하고 등급 및 담당자를 관리합니다.

고객 등록 +

고객명 또는 담당자 검색...

+ 등급

View

고객코드	고객명	등급	담당자	연락처	가입일	
C-0015	(주)테크인사이트	VIP	신재현	010-3434-5656	2026-05-20	...
C-0006	스마트러닝	일반	정혜원	010-7777-8888	2026-05-12	...
C-0011	에이치에스교육	VIP	조하운	010-8888-9999	2026-05-01	...
C-0012	(주)코딩클럽	일반	오세진	010-1212-3434	2026-04-15	...
C-0005	한국AI연구원	일반	최준호	010-5555-6666	2026-04-05	...
C-0009	(주)데이터랩	일반	윤아름	010-4444-5555	2026-03-18	...
C-0013	블루소프트	일반	서우진	010-5656-7878	2026-03-10	...
C-0004	(주)미래소프트	VIP	박민지	010-3333-4444	2026-03-01	...
C-0010	메타테크	일반	한승우	010-6666-7777	2026-02-25	...
C-0002	비티에스솔루션	일반	김남준	010-9876-5432	2026-02-10	...

10

Rows per page

Page 1 of 2

<< < 1 2 > >>

satnaing

satnaingdev@gmail.com

연 5 — Claude 하네스 이용(시연 이나 실습)

🎯 목표

하네스 사용법 및 리팩토링 확인

✅ 체크포인트

내부 소스코드가 리팩토링 되고 개선된다.

📄 프롬프트 / 명령어

1. `cli claude` 로그인
2. “현재 소스 전체 리팩토링 요소 확인해줘”

🆘 막혔을 때

- ① 페어 / 손

Q1. 가장 잘 된 것은?



현장 수집

Q2. 가장 답답했던 것은?



현장 수집 — 앰버 카드로 부착

그 답답함 — 기억한 채로, 5분만 쉬고 옹시다 ☕ (02:20–02:25)

😓 새 채팅마다
처음부터 설명

🎲 결과가 제각각

🔍 품질 검증은
결국 내 몫

“그 답답함, 전 세계 개발자가
똑같이 느꼈습니다.”

증거 — 지금 GitHub에서 가장 뜨거운 프로젝트들 →

설정과 프로세스가 곧 자산이 된 시대

🏆 1~3위 · 에이전트 자체의 진화

1 hermes-agent ★17.9만 (4개월) 자가개선 루프 · 절차를 스킬 문서로

2 openclaw ★37.7만 (총) 로컬 우선 개인 비서 · 생태계 중력

3 ECC 63 에이전트 · 249 스킬 멀티 하네스 통합 운영

△ 주: 일부 수치는 과장 가능성 — 교정된 값 기준 소개

🏆 4~7위 · 설정·인프라의 가치

4 superpowers 방법론의 코드화: 브레인스토밍→계획→TDD→리뷰

5 karpathy-skills ★16만 | 65줄짜리 CLAUDE.md | — agent-config 붐의 상징

6 agency-agents 14개 부서·200+ 에이전트 가상 조직

7 gstack YC CEO의 실사용 셋업 공개

🏆 8~10위 · 연구·디자인 자동화

8 autoresearch AI가 ML 실험 자가 반복 (단일 GPU 5분 학습)

9 awesome-design-md DESIGN.md 72개 브랜드 — 마크다운으로 UI 일관성

10 skills TDD·DDD를 에이전트 워크플로우에 이식

우리가 가져올 것 4가지

ECC

“에이전트 63개를 한 지붕 아래”
63 에이전트 · 249 스킬 · 멀티 하네스

가져올 것: 통합 운영 구조

superpowers

방법론의 코드화
브레인스토밍 → 계획 → TDD → 리뷰

가져올 것: 프로세스의 문서화

andrej-karpathy-skills

“코드 0줄, 마크다운 65줄이 ★16만”

가져올 것: 설정 문서 = 자산

gstack

YC CEO의 실사용 Claude Code 셋업

가져올 것: 검증된 실전 구성

“여러분이 느낀 문제 = 하네스가 푸는 문제”

여러분이 느낀 불편함

😓 새 채팅마다
처음부터 설명

🎲 결과가 제각각

🔍 품질 검증은
결국 내 몫

하네스가 푸는 문제

📁 컨텍스트 리셋 대응
(문서·메모리·스킬)

🔄 동일 프로세스 보장
(에이전트·워크플로우)

🚦 품질 유지 장치
(게이트 L1~L4 자동 검증)

하네스 = 이 세 장치를 하나로 묶은 시스템

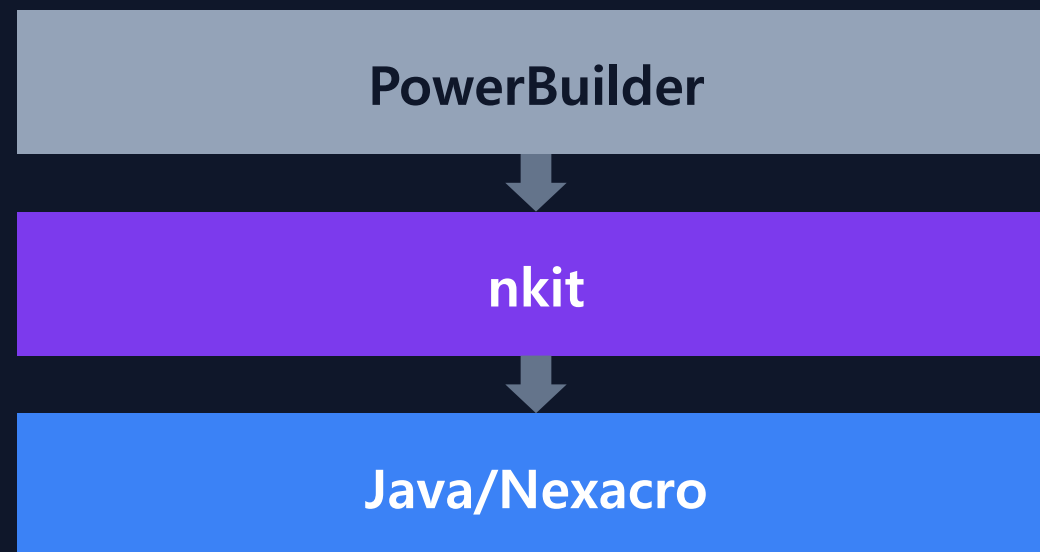
● 남의 이야기가 아닙니다 — 우리 회사의 nkit

ECC 기반으로 구성된 NDS의 AI 프레임워크

적용 실전: **PowerBuilder** → **Java/Nexacro** 마이그레이션

PDCA 엔진 + 품질 게이트 L1~L4 + Hook 통제

“레거시엔 안 통한다?” — 레거시가 바로 주 무대였습니다



신뢰는 느낌이 아니라 숫자로 만든다



Check 시 게이트 자동 실행 ↓



L3 실패 시 Act로 루프백 ↺ (점선)

통과 → 머지

“동작하지 않으면 나머지 점수는 무의미 — 그래서 **L3만 CRITICAL**”

AI로 AI 도구를 만들었습니다

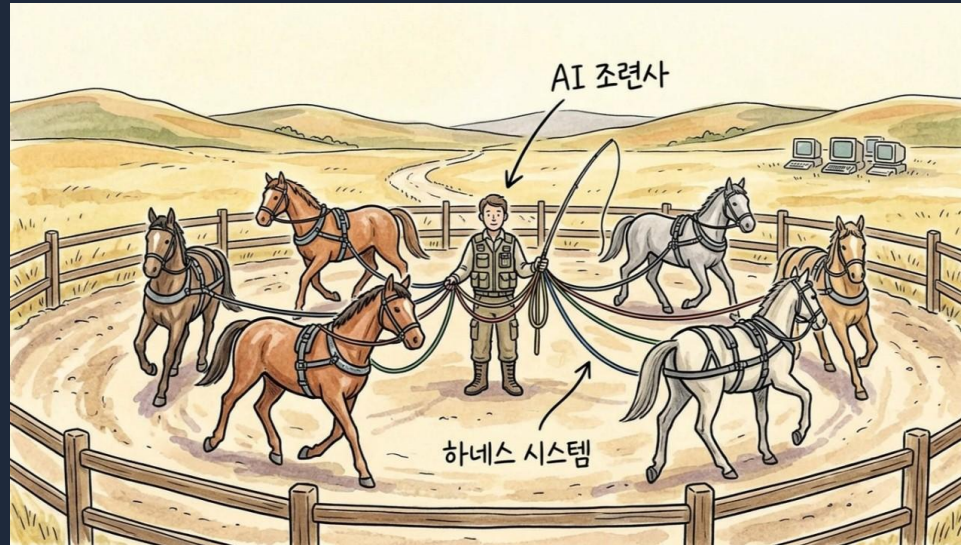


점선 경로: ECC · nkit → **"Claude로 분석"** → ai-edu

"여러분이 방금 클론한 실습 리포가 — 바로 이것입니다."

하네스는 완제품이 아니라 — 우리 팀 지식을 담은 그릇

① 운영·공통 프레임워크 분석
→ 절차를 md로



② .claude/skills/에 등록
← 팀 전체가 동일 절차 사용



같은 그림 — 이제 의미가 채워졌습니다

(다들 해본 것)

✓ 오늘 미션 1~4(+5)로 직접 체험

nkit · ai-edu = 우리 회사의 현재

+ 즉시 질의응답

1단계 채팅 (복불·1회성)

+ 코드베이스 접근
+ 반복 작업 가능
(1단계 장치 상속)

2단계 IDE (Antigravity)

+ 품질 게이트 L1~L4
+ 동일 프로세스 보장
+ 자동화(Hook·스킬)
(2단계 장치 상속)

3단계 하네스 (Claude Code CLI)

“채팅은 1회성, IDE는 반복, 하네스는 시스템 —
단계가 오를수록 신뢰 장치가 늘어난다.”

내일 아침, 채팅 한 단계 위에서 시작하세요

- ① 오늘 환경 그대로 — 본인 업무 코드에 미션 1(기존 화면 수정) 적용
- ② 설계서 md 한 장 써보기 — 65줄이면 충분합니다 ★
- ③ ai-edu 저장소 둘러보고 — 팀 스킬 후보 1개 제안

전부 개인 권한으로 오늘 당장 가능한 것들

Q&A

무엇이든 물어보세요

“신뢰는 도구가 아니라 **장치의 축적**에서 나옵니다”

감사합니다